

Analytic Geometry

CS115 - Math for Computer Science

September 9, 2024

Norm, Distance, Angle, Orthogonality

The definition of norm (or length), distance, angle, orthogonal, and normalizing for general inner product spaces closely parallel to those based on the dot product in Euclidean n -space

Norm (length) of $\mathbf{u}$: $\|\mathbf{u}\| = \sqrt{\langle \mathbf{u}, \mathbf{u} \rangle}$

Distance between $\mathbf{u}$ and $\mathbf{v}$: $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\| = \sqrt{\langle \mathbf{u} - \mathbf{v}, \mathbf{u} - \mathbf{v} \rangle}$

Angle between two nonzero vectors $\mathbf{u}$ and $\mathbf{v}$:

$$\cos \theta = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \|\mathbf{v}\|}, \quad 0 \leq \theta \leq \pi$$

Orthogonal: ($\mathbf{u} \perp \mathbf{v}$)

$\mathbf{u}$ and $\mathbf{v}$ are orthogonal if $\langle \mathbf{u}, \mathbf{v} \rangle = 0$

- Norm
- Inner Products
- Lengths and Distances
- Angles and Orthogonality
- Orthonormal Basis
- Orthogonal Complement
- Orthogonal Projections

Formal Definitions

- An inner product is a mapping $\langle \cdot, \cdot \rangle : V \times V \mapsto \mathbb{R}$ that satisfies the following conditions for all vectors $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all scalars $\lambda \in \mathbb{R}$:
 1. $\langle \mathbf{u} + \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{u}, \mathbf{w} \rangle + \langle \mathbf{v}, \mathbf{w} \rangle$
 2. $\langle \lambda \mathbf{v}, \mathbf{w} \rangle = \lambda \langle \mathbf{v}, \mathbf{w} \rangle$
 3. $\langle \mathbf{v}, \mathbf{w} \rangle = \langle \mathbf{w}, \mathbf{v} \rangle$
 4. $\langle \mathbf{v}, \mathbf{v} \rangle \geq 0$ and equal iff $\mathbf{v} = \mathbf{0}$
- The pair $(V, \langle \cdot, \cdot \rangle)$ is called an inner product space.

Examples

- **Example.** $V = \mathbb{R}^n$ and the dot product $\langle \mathbf{x}, \mathbf{y} \rangle := \mathbf{x}^T \mathbf{y}$
- **Example.** $V = \mathbb{R}^2$ and $\langle \mathbf{x}, \mathbf{y} \rangle := x_1 y_1 - (x_1 y_2 + x_2 y_1) + 2x_2 y_2$
- **Example.** $V = \{\text{continuous functions in } \mathbb{R} \text{ over } [a, b]\}$, $\langle u, v \rangle := \int_a^b u(x)v(x)dx$

Consider $\mathbb{R}^2$ with $\langle, \rangle$ defined for all $\mathbf{x}$ and $\mathbf{y}$ in $\mathbb{R}^2$ as

$$\langle \mathbf{x}, \mathbf{y} \rangle := \mathbf{x}^T \mathbf{A} \mathbf{y}, \quad \mathbf{A} := \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

Is $\langle, \rangle$ an inner product?

Let $\mathbf{x} = [x_1, x_2]^T$, $\mathbf{y} = [y_1, y_2]^T$. By direct computations, we have

$$\langle \mathbf{x}, \mathbf{y} \rangle = 2x_1y_1 + x_2y_1 + 2x_2y_2$$

$$\langle \mathbf{y}, \mathbf{x} \rangle = 2x_1y_1 + y_2x_1 + 2x_2y_2.$$

$\langle \mathbf{x}, \mathbf{y} \rangle \neq \langle \mathbf{y}, \mathbf{x} \rangle$. This implies that $\langle, \rangle$ is not an inner product.

$$\langle \mathbf{x}, \mathbf{y} \rangle = x_1 y_1 - (x_1 y_2 + x_2 y_1) + 2(x_2 y_2)$$

$$\langle \mathbf{y}, \mathbf{x} \rangle = y_1 x_1 - (y_1 x_2 + y_2 x_1) + 2(y_2 x_2).$$

$$\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$$

Let $\mathbf{z} = [z_1, z_2]^\top$, then for any real numbers a, b we have

$$\begin{aligned} \langle a\mathbf{x} + b\mathbf{y}, \mathbf{z} \rangle &= \langle [ax_1 + by_1, ax_2 + by_2]^\top, [z_1, z_2]^\top \rangle \\ &= (ax_1 + by_1)z_1 - (ax_1 + by_1)z_2 - (ax_2 + by_2)z_1 + 2(ax_2 + by_2)z_2 \\ &= (ax_1 z_1 - ax_1 z_2 - ax_2 z_1 + 2ax_2 z_2) + (by_1 z_1 - by_1 z_2 - by_2 z_1 + 2by_2 z_2) \\ &= a(x_1 z_1 - x_1 z_2 - x_2 z_1 + 2x_2 z_2) + b(y_1 z_1 - y_1 z_2 - y_2 z_1 + 2y_2 z_2) \\ &= a\langle \mathbf{x}, \mathbf{z} \rangle + b\langle \mathbf{y}, \mathbf{z} \rangle. \end{aligned}$$

$$\begin{aligned}\langle \mathbf{x}, \mathbf{x} \rangle &= x_1x_1 - (x_1x_2 + x_2x_1) + 2x_2x_2 \\ &= x_1^2 - 2x_1x_2 + 2x_2^2 = (x_1 - x_2)^2 + x_2^2,\end{aligned}$$

Norm

- A notion of the length of vectors
- **Definition.** A norm on a vector space V is a function $\|\cdot\| : V \mapsto \mathbb{R}$, such that for all $\lambda \in \mathbb{R}$ the following hold:
 - Absolutely homogeneous: $\|\lambda \mathbf{x}\| = |\lambda| \|\mathbf{x}\|$
 - Triangle inequality: $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$
 - Positive definite: $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| \iff \mathbf{x} = 0$

Example for $V \in \mathbb{R}^n$

- Manhattan Norm (also called ℓ_1 norm) For $\mathbf{x} = [x_1, \dots, x_n] \in \mathbb{R}^n$,

$$\|\mathbf{x}\|_1 := \sum_{i=1}^n |x_i|$$

- Euclidean Norm (also called ℓ_2 norm) For $\mathbf{x} \in \mathbb{R}^n$,

$$\|\mathbf{x}\|_2 := \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$$

Length

- Inner product naturally induces a norm by defining:

$$\|x\| := \sqrt{\langle x, x \rangle}$$

- Not every norm is induced by an inner product
- **Cachy-Schwarz inequality.** For the induced norm by the inner product,

$$|\langle x, y \rangle| \leq \|x\| \|y\|$$

Distance

- Now, we can introduce a notion of distance using a norm as:

Distance. $d(\mathbf{x}, \mathbf{y}) := \|\mathbf{x} - \mathbf{y}\| = \sqrt{\langle \mathbf{x} - \mathbf{y}, \mathbf{x} - \mathbf{y} \rangle}$

- If the dot product is used as an inner product in $\mathbb{R}^n$, it is **Euclidian distance**.
- Note.** The distance between two vectors does **NOT** necessarily require the notion of norm. Norm is just sufficient.
- Generally, if the following is satisfied, it is a suitable notion of distance, called **metric**.
 - Positive definite.** $d(\mathbf{x}, \mathbf{y}) \geq 0$ for all $\mathbf{x}, \mathbf{y}$ and $d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y}$
 - Symmetric.** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$
 - Triangle inequality.** $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$

Distance

Is the **distance** between two points **x**, **y** in $\mathbf{R}^n$???

... is **the length** of the vector from **x** to **y**

$$\text{dist}(x, y) = \|y - x\|$$

A **unit vector** is a vector **x** with length ???

$$\|x\| = \sqrt{x \cdot x} = 1$$

Fact. Let x be a nonzero vector in $\mathbf{R}^n$. The unit vector in the direction of x is the vector $x/\|x\|$

Compute the distance between

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} -1 \\ -1 \\ 0 \end{bmatrix}$$

using

a. $\langle \mathbf{x}, \mathbf{y} \rangle := \mathbf{x}^T \mathbf{y}$

b. $\langle \mathbf{x}, \mathbf{y} \rangle := \mathbf{x}^T \mathbf{A} \mathbf{y}, \mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & -1 \\ 0 & -1 & 2 \end{bmatrix}$

The distance between $\mathbf{x}$ and $\mathbf{y}$ is given by $\sqrt{\langle \mathbf{x} - \mathbf{y}, \mathbf{x} - \mathbf{y} \rangle}$ and we have

$$\mathbf{z} := \mathbf{x} - \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 3 \end{bmatrix}$$

a. In this case,

$$\langle \mathbf{z}, \mathbf{z} \rangle = \mathbf{z}^T \mathbf{z} = 2^2 + 3^2 + 3^2 = 22.$$

Hence the distance is $\sqrt{22}$.

The distance between $\mathbf{x}$ and $\mathbf{y}$ is given by $\sqrt{\langle \mathbf{x} - \mathbf{y}, \mathbf{x} - \mathbf{y} \rangle}$ and we have

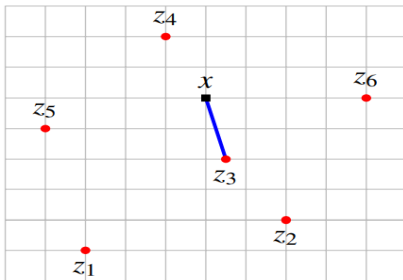
$$\mathbf{z} := \mathbf{x} - \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 3 \end{bmatrix}$$

$$\begin{aligned} \langle \mathbf{z}, \mathbf{z} \rangle &= \mathbf{z}^T \mathbf{A} \mathbf{z} \\ &= [2, 3, 3] \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 3 \end{bmatrix} \\ &= [7, 8, 3] \begin{bmatrix} 2 \\ 3 \\ 3 \end{bmatrix} = 47. \end{aligned}$$

Feature distance and nearest neighbors

- ▶ if x and y are feature vectors for two entities, $\|x - y\|$ is the *feature distance*
- ▶ if $z_1, \dots, z_m$ is a list of vectors, z_j is the *nearest neighbor* of x if

$$\|x - z_j\| \leq \|x - z_i\|, \quad i = 1, \dots, m$$

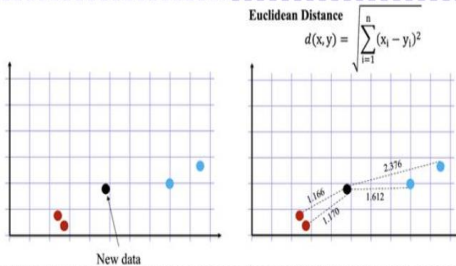


- ▶ these simple ideas are very widely used

K-Nearest Neighbors (KNN)

Step 1: Look at the data Step 2: Calculate distances Step 3: Find neighbours Step 4: Vote on labels

Classification



Ranking points

- 1 st
- 2 nd
- 3 rd
- 4 th

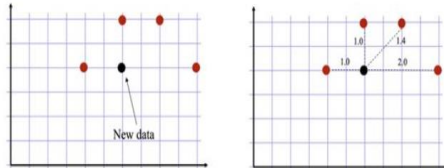
Find the nearest neighbours by ranking points by increasing distance

K=3 Nearest neighbours

- | | # of votes |
|--------------|------------|
| ● 1 st | 2 |
| ● 2 nd | |
| ● 3 rd | 1 |

Vote on the predicted class labels based on the class of the k nearest neighbours

Regression



Ranking points

- 1 st
- 2 nd
- 3 rd
- 4 th

Find the nearest neighbours by ranking points by increasing distance

K=4 Nearest neighbours

- 1 st
- 2 nd
- 3 rd
- 4 th

Compute the mean value of the k nearest neighbours

$$Y_{\text{pred}} = \frac{1}{k} \sum_{x \in \text{NB}} y_x$$

Document dissimilarity

- ▶ 5 Wikipedia articles: ‘Veterans Day’, ‘Memorial Day’, ‘Academy Awards’, ‘Golden Globe Awards’, ‘Super Bowl’
- ▶ word count histograms, dictionary of 4423 words
- ▶ pairwise distances shown below

	Veterans Day	Memorial Day	Academy Awards	Golden Globe Awards	Super Bowl
Veterans Day	0	0.095	0.130	0.153	0.170
Memorial Day	0.095	0	0.122	0.147	0.164
Academy A.	0.130	0.122	0	0.108	0.164
Golden Globe A.	0.153	0.147	0.108	0	0.181
Super Bowl	0.170	0.164	0.164	0.181	0

Image Classification: A core task in Computer Vision



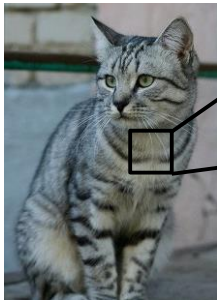
This image by NikitaJ is
licensed under [CC-BY 2.0](#)

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

The Problem: Semantic Gap



This image by Nikita is
 licensed under CC-BY 2.0

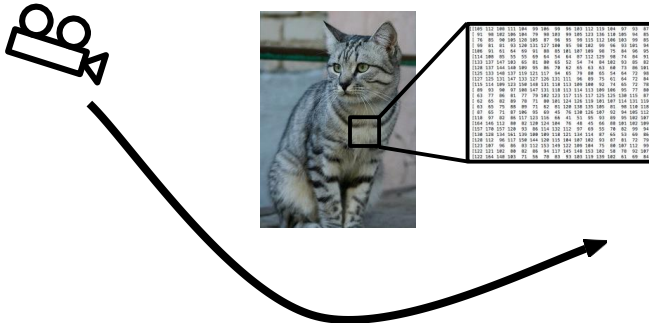
```
[185 112 180 111 104 99 106 99 96 103 112 119 104 97 93 87]
[ 91 90 182 106 104 79 98 103 99 105 121 116 118 105 94 85]
[ 76 85 90 105 120 105 87 96 95 99 115 112 106 103 99 85]
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]
186 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]
[114 100 85 55 55 69 64 54 64 87 112 129 98 74 84 91]
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]
[120 137 144 140 109 95 86 70 62 65 63 63 68 73 86 101]
125 133 140 137 119 121 117 94 65 70 88 65 54 64 72 90]
127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]
115 114 109 123 150 140 131 118 113 109 100 92 74 65 72 70]
[ 89 93 90 97 100 147 121 110 113 114 112 109 106 95 77 80]
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 110]
[ 63 65 75 88 89 71 62 81 120 130 135 105 81 90 110 110]
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]
110 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]
104 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]
157 170 127 120 93 86 114 132 112 97 69 55 78 82 99 94]
130 120 134 161 139 100 109 118 121 134 114 87 65 53 69 86]
120 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]
123 107 96 86 93 112 153 149 122 109 104 75 88 107 112 99]
122 121 102 80 82 86 94 117 145 140 153 102 58 78 92 107]
122 164 140 103 71 56 78 83 93 103 119 139 102 61 69 84]
```

What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
 (3 channels RGB)

Challenges: Viewpoint variation

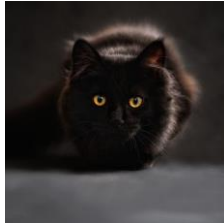


All pixels change when the camera moves!

Challenges: Illumination



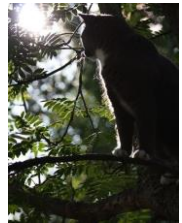
[This image is CC0 1.0 public domain](#)



[This image is CC0 1.0 public domain](#)

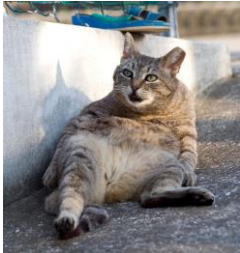


[This image is CC0 1.0 public domain](#)



[This image is CC0 1.0 public domain](#)

Challenges: Deformation



This image by Umberto Salvagnin
is licensed under [CC-BY 2.0](#)



This image by Umberto Salvagnin
is licensed under [CC-BY 2.0](#)



This image by sare bear is
licensed under [CC-BY 2.0](#)



This image by Tom Thai is
licensed under [CC-BY 2.0](#)

Challenges: Occlusion



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image by [jonsson](#) is licensed under [CC-BY 2.0](#)

Challenges: Background Clutter



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Intraclass variation



This image is [CC0 1.0](#) public domain

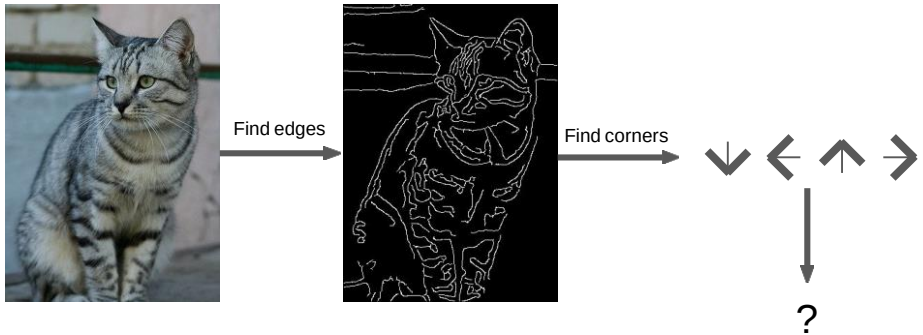
An image classifier

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

Attempts have been made



John Canny, "A Computational Approach to Edge Detection", IEEE TPAMI 1986

Data-Driven Approach

1. Collect a dataset of images and labels
2. Use Machine Learning to train a classifier
3. Evaluate the classifier on new images

Example training set

```
def train(images, labels):
    # Machine learning!
    return model
```

```
def predict(model, test_images):
    # Use model to predict labels
    return test_labels
```

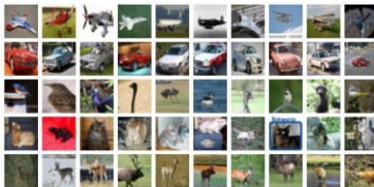
airplane

automobile

bird

cat

deer



First classifier: Nearest Neighbor

```
def train(images, labels):  
    # Machine learning!  
    return model
```



Memorize all
data and labels

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



Predict the label
of the most similar
training image

Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images



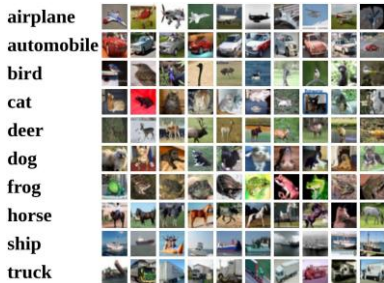
Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images



Test images and nearest neighbors



Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images", Technical Report, 2009.

Distance Metric to compare images

L1 distance:
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

-

pixel-wise absolute value differences

=

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

add → 456

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):  
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """  
        # the nearest neighbor classifier simply remembers all the training data  
        self.Xtr = X  
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """  
        num_test = X.shape[0]  
        # lets make sure that the output type matches the input type  
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)  
  
        # loop over all test rows  
        for i in xrange(num_test):  
            # find the nearest training image to the i'th test image  
            # using the L1 distance (sum of absolute value differences)  
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)  
            min_index = np.argmin(distances) # get the index with smallest distance  
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
    return Ypred
```

Nearest Neighbor classifier

Memorize training data

Nearest Neighbor classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

For each test image:
Find closest train image
Predict label of nearest image

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Q: With N examples,
how fast are training
and prediction?

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Q: With N examples,
how fast are training
and prediction?

A: Train $O(1)$,
predict $O(N)$


```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

A: Train $O(1)$,
predict $O(N)$

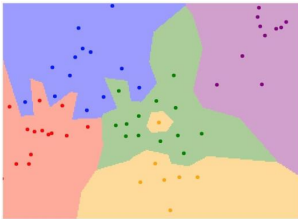
This is bad: we want classifiers that are **fast** at prediction; **slow** for training is ok

What does this look like?

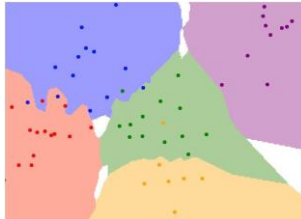


K-Nearest Neighbors

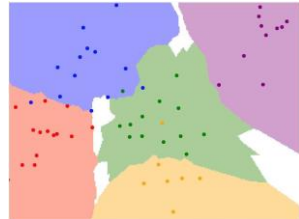
Instead of copying label from nearest neighbor,
 take **majority vote** from K closest points



K = 1



K = 3



K = 5

What does this look like?



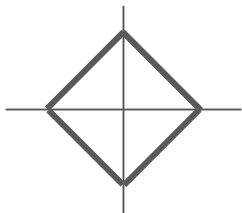
What does this look like?



K-Nearest Neighbors: Distance Metric

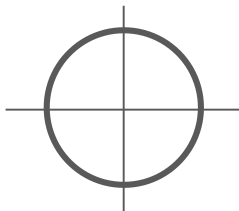
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

Hyperparameters

- What is the best value of **k** to use? What is the best **distance** to use?
- These are **hyperparameters**: choices about the algorithm that we set rather than learn

Hyperparameters

- What is the best value of **k** to use? What is the best **distance** to use?
- These are **hyperparameters**: choices about the algorithm that we set rather than learn
- Very problem-dependent.
- Must try them all out and see what works best.

Setting Hyperparameters

Idea #1: Choose hyperparameters
that work best on the data

Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data

Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data



train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data

Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

train

test

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better!

train

validation

test

Setting Hyperparameters

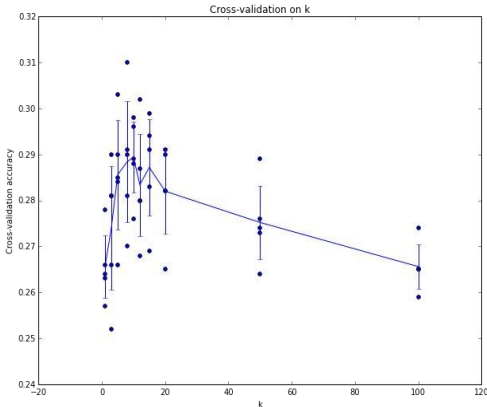
Your Dataset

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Useful for small datasets, but not used too frequently in deep learning

Setting Hyperparameters



Example of
5-fold cross-validation
for the value of k .

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that $k \approx 7$ works best
for this data)

k-Nearest Neighbor on images rarely used.

- Very slow at test time
- Distance metrics on pixels are not informative

Original



Boxed



Shifted



Tinted



(all 3 images have same L2 distance to the one on the left)

Original image/s
CC0 public domain

K-Nearest Neighbors: Summary

In **Image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**

The **K-Nearest Neighbors** classifier predicts labels based on nearest training examples

Distance metric and K are **hyperparameters**

Choose hyperparameters using the **validation set**; only run on the test set once at the very end!